Building2Building

Repository Review & Walkthrough Guide

A fair assessment of the refactored codebase

with hands-on exercises and known issues

Generated: April 2026

Table of Contents

1. Repository Overview & Architecture

Building2Building is an RL benchmark for building energy management using EnergyPlus simulations exposed through the Gymnasium interface. The repository has two main components:

- building2building/ - The core Python package (installable via pip)
- baselines/ - External experiment scripts (not part of the package)

Directory Structure (post-refactoring)

```
Building2Building/
|-- pyproject.toml          # Package definition (PEP 621)
|-- README.md
|-- .gitignore
|-- building2building/      # Main package
|   |-- __init__.py         # Public API exports
|   |-- api/                # new_make_env, list_buildings, etc.
|   |-- morphology.py       # Structured env representation
|   |-- scoring.py          # Normalized scoring
|   |-- benchmarks/         # 4 benchmark problems
|   |-- simulator/          # EnergyPlus integration & wrappers
|   |-- data/               # HuggingFace dataset registry
|   |-- pipeline/           # Actuator definitions
|   |-- config/             # Hydra config models & task presets
|   |-- types.py            # Reward, Task, Equipment types
|   |-- sources/            # Building data sources
|   `-- envs/               # Gym registration
|-- baselines/              # Experiment scripts
|   |-- configs/            # Hydra config groups
|   |-- controllers/        # Rule-based controllers
|   |-- models/             # Amorpheus transformer
|   |-- utils/              # Training, evaluation helpers
|   |-- plotting/           # Matplotlib figure scripts
|   |-- run_rule_based.py   # Baseline CSV generation
|   |-- train_ppo.py        # PPO specialist (Section 5)
|   |-- train_dynamics_adaptation.py  # Section 6.1
|   |-- train_cross_domain.py         # Section 6.2
|   |-- eval_*.py           # Evaluation scripts
|   |-- tune_controller.py  # Optuna controller tuning
|   `-- requirements.txt
|-- tests/                  # pytest suite (quick + long)
|-- jobs/                   # SLURM scripts (gitignored)
`-- scripts/                # Legacy scripts (gitignored)
```

Key design principle: baselines/ uses ONLY the public building2building API. Anyone using the benchmark should be able to write equivalent code.

2. Installation & Environment Setup

2.1 Core Package

```
# Clone and install in editable mode
git clone <repo-url> && cd Building2Building
pip install -e .

# With training dependencies (torch, SB3, hydra, optuna, wandb)
pip install -e '.[training]'

# With everything (training + test + dev + docs)
pip install -e '.[all]'
```

2.2 Baseline Dependencies

```
pip install -r baselines/requirements.txt
```

WARNING

baselines/requirements.txt lists only stable-baselines3, torch, wandb, tqdm. It is MISSING: hydra-core, omegaconf, optuna, matplotlib, pyyaml. These are covered by pip install -e '.[training]' from the main package, but if you install baselines separately, scripts will fail with ImportError.

2.3 EnergyPlus

EnergyPlus is required for running simulations. The package uses minergym (installed from Git) as the simulation backend. Set ENERGYPLUS_PATH if EnergyPlus is not on your system PATH.

```
export ENERGYPLUS_PATH=/path/to/EnergyPlus-24-1-0
# Optional: control dataset cache location
export STORE_PATH=/path/to/cache
```

2.4 Data Download

Building data is hosted on HuggingFace (vtaboga/building2building) and downloaded automatically on first use to ~/.cache/building2building/. The first call to new_make_env() or list_buildings() will trigger this.

3. The building2building Package (Core API)

3.1 Creating Environments

The primary entry point is b2b.new_make_env():

```
import building2building as b2b

# List available building types
types = b2b.list_building_types()
# ['OfficeSmall', 'OfficeMedium', 'RetailStandalone',
#  'RestaurantFastFood', 'Warehouse', 'SingleFamilyHouse']

# List building IDs for a type
train_ids = b2b.list_buildings('OfficeSmall', split='train')
test_ids  = b2b.list_buildings('OfficeSmall', split='test')

# Create an environment
env = b2b.new_make_env(
    'OfficeSmall',
    building_id=train_ids[0],  # or split='train', index=0
    task='task1',              # task1 through task4
)

obs, info = env.reset()
action = env.action_space.sample()
obs, reward, terminated, truncated, info = env.step(action)
env.close()
```

Key parameters of new_make_env:
- building_type: str (one of the 6 types)
- split: 'train' or 'test'
- index: int (position in the split list)
- building_id: str (alternative to split+index)
- task: str ('task1'-'task4') or TaskPreset dict
- run_period: 'full_year' (default)
- timesteps_per_hour: 12 (default, i.e. 5-min steps)

3.2 Observation & Action Spaces

Each environment exposes rich metadata through env.metadata:

```
env = b2b.new_make_env('OfficeSmall', task='task1')

# Named observations and actions
obs_names = env.metadata['observation_names']  # list[str]
act_names = env.metadata['action_names']       # list[str]

# HVAC equipment descriptions
equipment = env.metadata['hvac_equipment']  # list[Equipment]

# Building metadata
area = env.metadata['total_conditioned_area']
zones = env.metadata['zone_names']
```

```
print(f'Obs dim: {env.observation_space.shape[0]}')
print(f'Act dim: {env.action_space.shape[0]}')
print(f'Obs names: {obs_names[:5]}...')
print(f'Act names: {act_names[:3]}...')
```

Observation dimensions vary by building type (different zone counts, equipment). Action dimensions also vary. This heterogeneity is a core challenge of the benchmark.

3.3 Morphology Graph (Structured Representation)

The morphology graph (Section 3.4 of the paper) provides a structured decomposition of the flat observation/action vectors into per-node local spaces. Each node has a type (weather, calendar, energy, zone variants, supply) and maps to specific indices in the flat vectors.

```
env = b2b.new_make_env('OfficeSmall', task='task1')
morph = env.metadata['morphology']  # b2b.Morphology

# Inspect the graph
print(f'Nodes: {len(morph.nodes)}')
print(f'Edges: {len(morph.edges)}')
print(f'Type counts: {morph.type_counts()}')

# Split flat observation into per-node dicts
obs, _ = env.reset()
local_obs = morph.split_observation(obs)
# Returns dict[str, np.ndarray] keyed by node.node_id

# Join per-node actions back to flat vector
actions_dict = {n.node_id: np.zeros(n.node_type.action_dim)
                for n in morph.nodes if n.node_type.action_dim > 0}
flat_action = morph.join_actions(actions_dict)

env.close()
```

Node types defined in building2building.morphology:
WEATHER (obs=2, act=0), CALENDAR (obs=2, act=0), ENERGY (obs=1, act=0),
UNITARY_ZONE, VAV_ZONE, VAV_SUPPLY, HEATING_ZONE, UNCONTROLLED_ZONE
Each with specific observation/action dimensions.

3.4 Benchmarks

Four benchmark problems correspond to the paper's generalization axes:

```
import building2building as b2b

# 1. Dynamics Adaptation (Section 6.1)
#    Same building type, different instances
bench = b2b.benchmarks.DynamicsAdaptation(
    difficulty='easy',  # easy/medium/hard
    task='task1',
)
train_ids = bench.train_building_ids()  # ~900 buildings
test_ids  = bench.test_building_ids()   # ~71 buildings
# Difficulty mapping:
#   easy   -> SingleFamilyHouse (action_dim=2)
#   medium -> OfficeSmall       (action_dim=10)
```

```
#   hard   -> OfficeMedium      (action_dim=33)

# 2. Cross-Domain Generalization (Section 6.2)
#    Train on one type, test on a different type
bench2 = b2b.benchmarks.CrossDomainGeneralization(
    difficulty='easy',  # easy/medium/hard
)
train_envs = bench2.make_train_envs(n=3)
test_envs  = bench2.make_test_envs(n=3)
# Difficulty mapping:
#   easy   -> Retail -> OfficeSmall
#   medium -> Retail -> Warehouse
#   hard   -> OfficeSmall -> OfficeMedium

# 3. Goal Adaptation
bench3 = b2b.benchmarks.GoalAdaptation(
    train_task='task1', test_task='task2'
)

# 4. Action Space Transfer
bench4 = b2b.benchmarks.ActionSpaceTransfer(
    system_type='unitary', direction='expand'
)
```

3.5 Scoring

Normalized scoring compares agent performance to the rule-based baseline. It reads baseline_returns.csv from the repo root.

```
score = b2b.compute_normalized_score(
    cumulative_return=-5000.0,  # FIRST argument
    building_type='OfficeSmall',
    task='task1',
    building_id=42,  # optional
)
# Returns cumulative_return / baseline_return
# Score > 1.0 means better than baseline
```

WARNING

IMPORTANT: The signature is compute_normalized_score(cumulative_return, building_type, task, building_id=None). The first positional argument is the return value, NOT the building type. Some eval scripts have this wrong (see Known Bugs section).

3.6 Wrappers for Multi-Building Training

```
import building2building as b2b

env = b2b.new_make_env('OfficeSmall', task='task1')

# 1. Pad observations to uniform size across buildings
env = b2b.PadObservation(env, target_size=20)

# 2. Running-mean normalize observations
env = b2b.NormalizeObservation(env)

# 3. Append building parameters to observations
env = b2b.AugmentObservationWithBuildingParams(env)
```

```
# 4. Resample building on each reset
def factory(idx: int) -> gym.Env:
    return b2b.new_make_env('OfficeSmall', split='train',
                            index=idx, task='task1')
env = b2b.ResampleBuildingOnResetWrapper(
    factory, available_indices=[0, 1, 2, 3]
)
```

4. The baselines/ Directory

4.1 Hydra Configuration System

All training scripts use Hydra for configuration. The config structure is:

```
baselines/configs/
|-- config.yaml              # Root: defaults + seed + output_dir
|-- experiment/              # Per-script settings
|   |-- eval_rule_based.yaml
|   |-- train_ppo.yaml
|   |-- train_dynamics_specialist.yaml
|   |-- train_dynamics_baseline.yaml
|   |-- train_dynamics_parameterized.yaml
|   |-- train_cross_domain.yaml
|   `-- tune_controller.yaml
|-- policy/                  # Algorithm hyperparameters
|   |-- ppo.yaml             # Table 5 defaults
|   |-- ppo_parameterized.yaml
|   |-- unitary_g36.yaml     # Controller defaults
|   `-- ashrae_air_loop.yaml
|-- reward/                  # Task definitions
|   |-- task1.yaml ... task4.yaml
|-- training/default.yaml    # total_timesteps, n_envs, etc.
|-- tuned_controllers/       # 45 Optuna-tuned controller YAMLs
`-- wandb/default.yaml       # W&B logging config
```

Usage pattern:

```
# Every script follows this pattern:
python -m baselines.<script> experiment=<name> [overrides...]

# Examples:
python -m baselines.run_rule_based experiment=eval_rule_based
python -m baselines.train_ppo experiment=train_ppo seed=42
python -m baselines.train_dynamics_adaptation \
    experiment=train_dynamics_parameterized difficulty=medium
```

4.2 Rule-Based Controllers

Two controllers in baselines/controllers/:
- UnitaryG36Policy: For single-zone packaged systems (5 building types)
- AshraeAirLoopPolicy: For VAV multi-zone systems (OfficeMedium only)

Both require bind_env(env) to wire observation/action indices from metadata. They implement predict(obs, deterministic=True) for compatibility with run_episode().

```
from baselines.controllers.unitary_g36 import (
    UnitaryG36Config, UnitaryG36Policy
)
from baselines.utils.evaluation import run_episode

env = b2b.new_make_env('OfficeSmall', task='task1')
policy = UnitaryG36Policy(UnitaryG36Config())
```

```
policy.bind_env(env)
result = run_episode(env, policy)
print(f'Reward: {result.total_reward:.1f}')
env.close()
```

4.3 PPO Specialist Training (Section 5)

train_ppo.py trains one PPO per (building_type, building_id, task) combination. Uses SB3's PPO with the paper's hyperparameters from configs/policy/ppo.yaml.

```
# Train on all types and tasks (full paper experiment)
python -m baselines.train_ppo experiment=train_ppo

# Quick test: one type, one task, short training
python -m baselines.train_ppo experiment=train_ppo \
    building_types=[OfficeSmall] tasks=[task1] \
    buildings_per_type=1 training.total_timesteps=10000

# Models saved to: outputs/train_ppo/<date>/<time>/
#   models/<building_type>/<task>/ppo_<building_id>.zip
#   results.csv
```

4.4 Dynamics Adaptation (Section 6.1)

Three training approaches on the DynamicsAdaptation benchmark:

- specialist: Independent PPO per building (1 env, no wrappers)
- baseline: Single PPO across all train buildings (PadObs + NormObs)
- parameterized: Same + AugmentObservationWithBuildingParams

```
# Specialist approach
python -m baselines.train_dynamics_adaptation \
    experiment=train_dynamics_specialist

# Parameterized approach (the paper's main result)
python -m baselines.train_dynamics_adaptation \
    experiment=train_dynamics_parameterized difficulty=easy
```

4.5 Cross-Domain Transfer (Section 6.2)

Trains the Amorpheus type-heterogeneous transformer policy. Uses a custom PPO training loop (not SB3) to handle the morphology-based split/join operations.

```
python -m baselines.train_cross_domain experiment=train_cross_domain

# The Amorpheus architecture:
# 1. Per-node-type encoders (local obs -> embedding)
# 2. Type embeddings added to each node
# 3. Transformer encoder aggregates across nodes
# 4. Per-node-type decoders (embedding -> local action)
# 5. Value head from mean pooled embeddings
```

4.6 Evaluation Scripts

```
# Evaluate PPO specialists
python -m baselines.eval_ppo \
```

```
    --model-dir outputs/train_ppo/.../models

# Evaluate dynamics adaptation
python -m baselines.eval_dynamics_adaptation \
    --model-path outputs/.../models/multi_parameterized.zip \
    --difficulty easy --approach parameterized

# Evaluate cross-domain transfer
python -m baselines.eval_cross_domain \
    --model-path outputs/.../amorpheus_policy.pt \
    --test-building-types Warehouse SingleFamilyHouse
```

4.7 Controller Tuning

```
python -m baselines.tune_controller experiment=tune_controller \
    building_type=OfficeSmall climate_zone=1 n_trials=50
```

4.8 Plotting

```
python -m baselines.plotting.plot_ppo_specialist \
    --ppo-csv results_ppo.csv --baseline-csv baseline_returns.csv

python -m baselines.plotting.plot_dynamics_adaptation \
    --specialist-csv ... --baseline-csv ... --parameterized-csv ...

python -m baselines.plotting.plot_cross_domain \
    --results-csv results_cross_domain.csv
```

5. Known Bugs & Issues (Honest Assessment)

This section documents all verified and potential issues found during the review. They are categorized by severity.

5.1 Confirmed Bugs

BUG: compute_normalized_score argument order in eval scripts

In eval_ppo.py (line 58) and eval_dynamics_adaptation.py (line 68), the call is:
  b2b.compute_normalized_score(building_type, ep.total_reward, task=task)
But the correct signature is:
  compute_normalized_score(cumulative_return, building_type, task, ...)
The first two positional arguments are SWAPPED. This means:
  - The function receives a string as cumulative_return
  - It will likely raise a TypeError or produce nonsensical results
  - The normalized scores in eval output CSVs will be wrong
FIX: Swap to b2b.compute_normalized_score(ep.total_reward, building_type, task=task)
NOTE: train_ppo.py (line 106) has the CORRECT order.

BUG: eval_ppo.py model path/name mismatch with train_ppo.py

train_ppo.py saves models to:
  outputs/.../models/{building_type}/{task}/ppo_{building_id}.zip
eval_ppo.py expects a FLAT directory with files named:
  ppo_{building_type}_{building_id}_{task}.zip
The _parse_model_filename() function splits on underscores in a way
that cannot handle the nested directory structure.
FIX: Either change eval_ppo to walk subdirectories, or change train_ppo
to save with the expected flat naming convention.

BUG: plot_ppo_specialist.py CSV schema mismatch

plot_ppo_specialist.py uses load_results_csv() from common.py which
expects columns: building_type, task, building_id, reward_mean.
But eval_ppo.py outputs: building_type, building_id, task, reward,
episode_length, normalized_score (NO reward_mean column).
FIX: Either add a reward_mean column to eval_ppo output, or update
the plotting script to use the 'reward' column.

5.2 Missing Dependencies

WARNING

baselines/requirements.txt is incomplete. It lists stable-baselines3, torch, wandb, tqdm but is MISSING:
  - hydra-core (all training scripts use @hydra.main)
  - omegaconf (used in every Hydra script)
  - optuna (tune_controller.py)
  - matplotlib (plotting scripts)
  - pyyaml (run_rule_based.py loads YAML configs)
These ARE included in the main package's [training] extra, so 'pip install -e .[training]' covers them. But requirements.txt alone is insufficient.

5.3 Potential Issues & Edge Cases

WARNING

tune_controller.py: _make_objective() return type annotation says 'optuna.Trial' but it returns a Callable[[optuna.Trial], float]. Functionally harmless but misleading for type checkers.

WARNING

eval_dynamics_adaptation.py: Hard-codes PadObservation(env, target_size=20) for test evaluation. But training in train_dynamics_adaptation.py uses _detect_max_obs_dim() which may compute a different value. If the trained model expects a different observation dimension, inference will fail or produce garbage.

WARNING

eval_cross_domain.py: Uses torch.load(..., weights_only=True) which requires PyTorch >= 2.4. Older versions will raise a TypeError.

WARNING

scoring.py: Reads baseline_returns.csv from Path(__file__).parent.parent, which resolves to the repo root. If the file doesn't exist (it must be generated first by running run_rule_based.py), compute_normalized_score raises FileNotFoundError.

WARNING

Hydra experiment configs use @package _global_ which merges keys into the top-level config. Some experiment configs define 'training:' overrides that merge with (not replace) the defaults. This is correct Hydra behavior but can be confusing.

WARNING

train_cross_domain.py: The rollout collection doesn't properly handle episode boundaries within a rollout. The GAE computation uses dones flags but the environment is auto-reset, meaning the value bootstrap at episode boundaries may be incorrect (common simplification in research code).

5.4 Test Coverage Gaps

The existing pytest suite (tests/) covers the core building2building package well for quick tests (API, registry, scoring, types, benchmarks, config). However:

- NO tests exist for baselines/ code (controllers, training, eval)
- Long tests require EnergyPlus and network access (HuggingFace dataset)
- No integration test for the full train -> eval -> plot pipeline
- No test for Hydra config composition (experiment + policy + reward)

5.5 Documentation Gaps

- Root README.md still references deleted scripts/training/ paths
- No docstring for several internal functions in the baselines
- No API reference docs generated (mkdocs configured but not built)

6. Hands-On Exercises

These exercises walk you through the repository from simple to complex. Each exercise has a concrete goal and expected output. Do them in order.

Exercise 1: Verify the installation

Goal: Confirm the package imports correctly and data is accessible.

Steps:
1. Run: python -c "import building2building as b2b; print(b2b.list_building_types())"
2. Run: python -c "import building2building as b2b; print(b2b.list_buildings('OfficeSmall', split='test')[:3])"
3. Run the quick test suite: pytest -m quick

Expected:
- Step 1 prints 6 building types
- Step 2 prints 3 building IDs (strings)
- Step 3: all tests pass (some may skip if fixtures missing)

If step 2 fails with a download error, check your network and HuggingFace access.

Exercise 2: Create and inspect an environment

Goal: Understand the observation/action space and metadata.

Write a script that:
1. Creates an OfficeSmall env with task1
2. Prints observation_space.shape, action_space.shape
3. Prints the first 5 observation names and all action names
4. Prints the HVAC equipment types
5. Resets, takes 3 random steps, prints rewards
6. Closes the env

Template:
```
import building2building as b2b
env = b2b.new_make_env('OfficeSmall', split='test', index=0, task='task1')
# ... your code here ...
env.close()
```

Verify: obs_dim should be ~17, act_dim should be ~10 for OfficeSmall.

Exercise 3: Explore the morphology graph

Goal: Understand the structured environment representation.

Write a script that:
1. Creates an OfficeSmall env
2. Gets morph = env.metadata['morphology']
3. Prints number of nodes, edges, type_counts()
4. For each node, prints: node_id, node_type.name, obs_indices, action_indices
5. Resets the env, calls morph.split_observation(obs)
6. Prints the shape of each local observation
7. Verifies the total obs indices match the observation space

Then repeat for OfficeMedium and compare the graphs.

Key question: How do node types differ between unitary and VAV systems?

Exercise 4: Run a rule-based controller manually

Goal: Understand how controllers work and verify reward magnitudes.

Steps:
1. Create an OfficeSmall env with task1
2. Instantiate UnitaryG36Policy with default config
3. Call policy.bind_env(env)
4. Use run_episode(env, policy) from baselines.utils.evaluation
5. Print total_reward and episode_length
6. Repeat for OfficeMedium (use AshraeAirLoopPolicy)

Expected: Rewards will be large negative numbers (cost-based).
Typical range: -5000 to -50000 depending on building type.

Exercise 5: Run the Hydra-based rule-based evaluation

Goal: Verify the Hydra configuration pipeline works end-to-end.

Steps:
1. Run with a minimal scope:
   python -m baselines.run_rule_based experiment=eval_rule_based \
     building_types=[OfficeSmall] tasks=[task1] max_buildings_per_type=1
2. Check that baseline_returns.csv is generated
3. Inspect the CSV: does it have the right columns?
   (building_type, task, building_id, reward_run1, reward_mean)
4. Try overriding the output path:
   python -m baselines.run_rule_based experiment=eval_rule_based \
     building_types=[OfficeSmall] tasks=[task1] \
     max_buildings_per_type=1 output_csv=test_output.csv

Watch for: Hydra errors about missing keys, incorrect config composition.

Exercise 6: Train a quick PPO specialist

Goal: Verify the training pipeline works.

Steps:
1. First ensure baseline_returns.csv exists (from Exercise 5)
2. Run a very short training:
   python -m baselines.train_ppo experiment=train_ppo \
     building_types=[OfficeSmall] tasks=[task1] \
     buildings_per_type=1 training.total_timesteps=1000 \
     training.n_envs=1
3. Check that outputs/ contains:
   - models/OfficeSmall/task1/ppo_<id>.zip
   - results.csv
4. Inspect results.csv: does it have normalized_score?

Warning: This will be slow if EnergyPlus runs a full year.
The script doesn't have a quick-test mode for episode length.

Exercise 7: Verify the compute_normalized_score bug

Goal: Confirm the argument-order bug in eval scripts.

Steps:
1. Read baselines/eval_ppo.py line 58-59
2. Read baselines/eval_dynamics_adaptation.py line 68-69
3. Read building2building/scoring.py line 75-79
4. Compare: is the first positional argument the return or the type?
5. Read baselines/train_ppo.py line 106-111 for the correct usage

Expected finding: eval scripts pass (building_type, reward) but the
function expects (cumulative_return, building_type). The arguments are swapped.

Bonus: Fix the bug and verify with a simple test.

Exercise 8: Verify eval_ppo model path discovery

Goal: Confirm the path mismatch between training and evaluation.

Steps:
1. After Exercise 6, look at the saved model path structure:
   ls -R outputs/train_ppo/.../models/
2. Read eval_ppo.py's _parse_model_filename() function
3. Does it expect: ppo_<type>_<id>_<task>.zip ?
4. Does the training save: models/<type>/<task>/ppo_<id>.zip ?
5. Can eval_ppo discover the models? (Hint: No.)

Bonus: Write a fix for eval_ppo that walks subdirectories.

Exercise 9: Test the dynamics adaptation benchmark

Goal: Verify the DynamicsAdaptation benchmark API.

Steps:
1. Instantiate DynamicsAdaptation(difficulty='easy')
2. Print building_type, len(train_ids), len(test_ids)
3. Create one train env and one test env
4. Compare their observation/action spaces
5. Run a random-action episode on the train env
6. Close both envs

Key question: Are obs dims the same for all buildings of the same type?

Exercise 10: Inspect the Amorpheus architecture

Goal: Understand the cross-domain policy network.

Steps:
1. Read baselines/models/amorpheus.py
2. Create an OfficeSmall env, get its morphology
3. Instantiate AmorpheusPolicy(morphology=morph)

4. Count total parameters
5. Create a dummy obs tensor and call policy(obs)
6. Check that output shapes match (batch, action_dim) and (batch, 1)
7. Now get a morphology from OfficeMedium
8. Set policy.morphology = new_morph and call again
9. Verify: the SAME policy works on different building types

Template:

```
import torch
from baselines.models.amorpheus import AmorpheusPolicy
policy = AmorpheusPolicy(morphology=morph, embed_dim=64)
obs_t = torch.randn(1, env.observation_space.shape[0])
actions, values = policy(obs_t)
print(actions.shape, values.shape)
```

Exercise 11: End-to-end mini experiment

Goal: Run a complete (tiny) experiment pipeline.

Steps:

1. Generate baseline CSV for 1 building:

```
python -m baselines.run_rule_based experiment=eval_rule_based \
    building_types=[SingleFamilyHouse] tasks=[task1] \
    max_buildings_per_type=1
```

2. Train a specialist for 2000 steps:

```
python -m baselines.train_ppo experiment=train_ppo \
    building_types=[SingleFamilyHouse] tasks=[task1] \
    buildings_per_type=1 training.total_timesteps=2000 \
    training.n_envs=1
```

3. Check that results.csv has a normalized_score
4. Compare PPO reward to baseline reward from step 1

Expected: After only 2000 steps, PPO will likely be worse than baseline.
The point is to verify the pipeline, not achieve good results.

Exercise 12: Audit the requirements.txt

Goal: Verify all imports are satisfiable.

Steps:

1. For each baselines/*.py file, list all import statements
2. Cross-reference with baselines/requirements.txt
3. Cross-reference with pyproject.toml [project.optional-dependencies]
4. Identify any package imported but not listed anywhere

Expected findings:

- hydra, omegaconf: in pyproject.toml [training] but not requirements.txt
- optuna: in pyproject.toml [training] but not requirements.txt
- matplotlib: NOT in either (but used by plotting scripts)
- pyyaml: NOT explicitly listed (but is a transitive dep of hydra)

7. Verification Checklist

Use this checklist to systematically verify the repository state. Mark each item as you go.

7.1 Package Integrity

- [ ] import building2building as b2b succeeds
- [ ] b2b.list_building_types() returns 6 types
- [ ] b2b.list_buildings() returns non-empty lists for all types
- [ ] b2b.new_make_env() creates a valid Gymnasium environment
- [ ] env.metadata contains: observation_names, action_names, morphology, hvac_equipment
- [ ] pytest -m quick passes (core package tests)

7.2 Morphology

- [ ] Morphology is in env.metadata['morphology'] for all building types
- [ ] split_observation() returns dict with one entry per node
- [ ] join_actions() produces array matching action_space.shape
- [ ] Node types match expected equipment (unitary zones, VAV zones, etc.)

7.3 Benchmarks

- [ ] DynamicsAdaptation(difficulty='easy') instantiates correctly
- [ ] DynamicsAdaptation train_building_ids() returns non-empty list
- [ ] CrossDomainGeneralization(difficulty='easy') instantiates correctly
- [ ] CrossDomainGeneralization make_train_envs() creates environments
- [ ] GoalAdaptation and ActionSpaceTransfer instantiate

7.4 Baselines - Configuration

- [ ] Hydra config.yaml loads without errors (test: python -c 'from omegaconf import OmegaConf')
- [ ] All experiment/*.yaml files are valid YAML
- [ ] All 45 tuned_controllers/*.yaml files are valid YAML
- [ ] Config composition works: experiment + policy + reward + training + wandb

7.5 Baselines - Scripts

- [ ] run_rule_based.py runs with experiment=eval_rule_based (1 building)
- [ ] train_ppo.py runs with experiment=train_ppo (1 building, 1000 steps)
- [ ] train_dynamics_adaptation.py accepts all 3 experiment configs
- [ ] train_cross_domain.py runs with experiment=train_cross_domain
- [ ] tune_controller.py runs with experiment=tune_controller (5 trials)

7.6 Known Bugs (to fix)

- [ ] eval_ppo.py: compute_normalized_score argument order FIXED
- [ ] eval_dynamics_adaptation.py: compute_normalized_score argument order FIXED

- [ ] eval_ppo.py: model path discovery matches train_ppo.py save paths
- [ ] plot_ppo_specialist.py: CSV schema matches eval_ppo.py output
- [ ] baselines/requirements.txt: hydra-core, omegaconf, optuna, matplotlib added
- [ ] tune_controller.py: _make_objective return type annotation fixed
- [ ] eval_dynamics_adaptation.py: hard-coded pad size matches training

7.7 Documentation

- [ ] baselines/README.md is up-to-date with current script usage
- [ ] Root README.md updated (removed references to deleted scripts/)
- [ ] All public functions have type hints
- [ ] Hydra configs have comment headers explaining their purpose